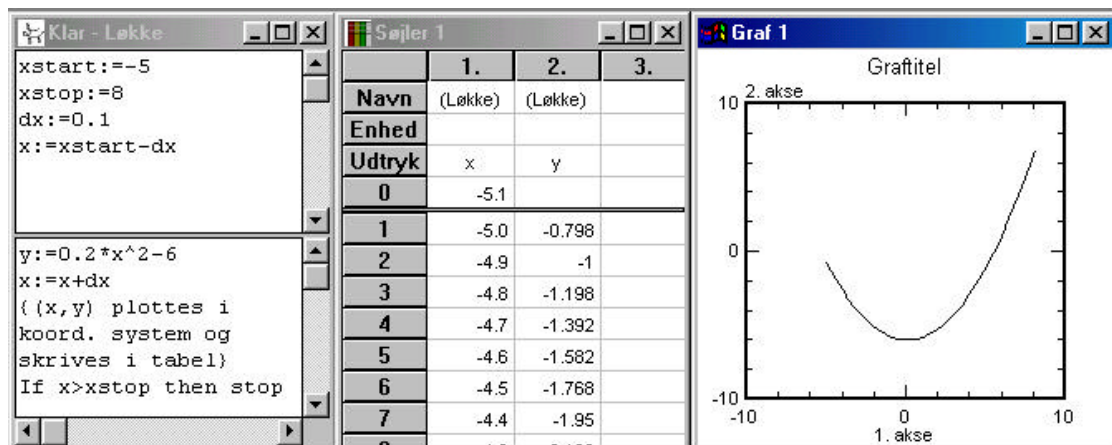


3: Variabel-typer. Klar-Løkke vinduet. Erklæring af funktion.



Figur 3.1: Udskrift fra MAT3a.FPR. Programmet tabellægger funktionen $y = \frac{1}{5}x^2 - 6$ fra $x = -5$ til $+8$, og tegner samtidigt grafen. Opbygningen starter med indskrivning i Klar-Løkke vinduet. Afviklingen startes med de sidste af de tre båndoptager-symboler

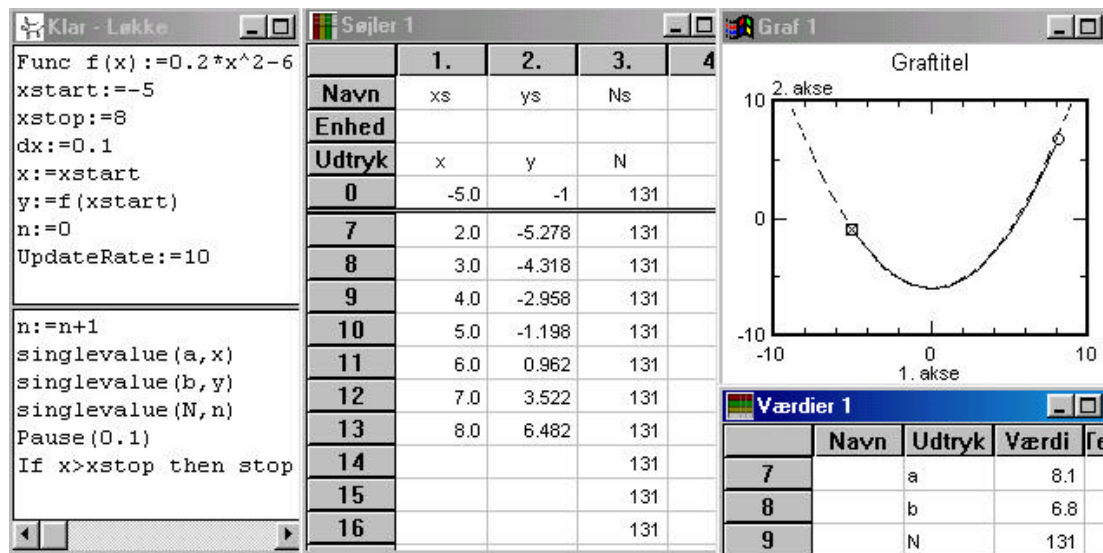


der står for : Reset - Stop - Kør

Programmet vil så først gennemløbe Klar-delen (den øverste). Derpå vil det gennemløbe Løkke-delen gentagne gange, indtil den første af to stop-betingelser nås: Den ene af disse er den i løkken angivne $x > x_{stop}$. Den anden er en indbygget sikkerhed i form af en øvre grænse for antal gennemløb, en grænse, der kan justeres via menupunktet afvikling/stopbetingelse.

Bemærk at første x i første søjle starter et skridt før x_{start} - og stadig viser sin værdi efter reset. Bemærk også at programforløbet ikke påvirkes af kommentaren $\{(x,y) \text{ plottes } \dots\}$, der starter med “{” og afsluttes med “}”

Fpro-vejledningen skelner mellem 3 forskellige typer af variable: *Enkeltværdi-variable*, *søjlevariable* og *løkkevariable*. De i (enkelt)værdi-vinduet fra fig. 2.2 oprettede x_{min} , x_{max} , y_{min} og y_{max} er eksempler på enkeltværdi-variable, medens x og y i fig. 2.3 er eksempler på søjlevariable. Variable som x og y i fig. 3.1, der overskrives med en ny værdi ved hvert gennemløb af løkken er naturligvis løkke-variable. Havde vi givet første søjle navnet u , da ville vi have brugt enkeltværdi-variablen x til at definere søjlevariablen u . Visse variable falder ikke så umiddelbart i een af de nævnte grupper. Eksemplet i fig. D2.1 definerer da vistnok en særlig *grafvariabel* t . I eksemplet i fig. 3.3 overfører *singlevalue(a,x)* i Klar-Løkke vinduet værdien af løkke-variablen x til enkeltværdi-variablen a . Endvidere demonstreres, at en enkeltværdi-variabel åbenbart kan skifte værdi under programafviklingen. At den alligevel ikke kan opfattes som en løkke-variabel, fremgår hurtigt, når man ser, hvad programmet laver.



	1. koordinat	2. koordinat	Signatur
1. graf	xs	ys	
2. graf	a	b	○
3. graf	xstart	f(xstart)	□
4. graf	xstart	f(xstart)	×
5. graf	t	f(t)	---

Figur 3.2: Udskrift fra MAT3b.FPR. Dette lille program er lavet blot for at illustrere nogle muligheder. Efter *Reset* viser grafen os den punkterede parabel, med det lille gennemkrydsede kvadrat placeret i punktet $(-5, -1)$. Ved *Kør* begynder den lille cirkel at vandre langs parabeln, idet den efterlader den fuldt optrukne streg bag sig. Efter hvert lille ryk fremad udskrives den ny koordinater (a, b) i *Værdier* sammen med det antal gange, løkken har været gennemløbet.

Bemærkning 1: Ordren *pause(0.1)* sinker løkkens gennemløb med 0.1 sek. Med *UpdateRate:= 10* vil løkken gentage sig selv 10 gange, før den tilføjer næste grafpunkt og tilføjer næste værdi til søjlevariablerne xs og ys . Søjlevariablen N får tildelt enkeltværdien n hele vejen ned. Og idet punktet (a, b) plottes svarende til de opskrevne værdier af enkeltværdi-variablerne a og b , så forsvinder den foregående plotning.

Bemærkning 2: Vi har defineret funktionen f ved forskriften $f(x) = 0.2x^2 - 6$. Vi kunne imidlertid lige så godt have skrevet f. eks. $f(t) = 0.2t^2 - 6$. Vi må kalde den uafhængige variabel, hvad vi vil. Vi kan således også skrive $f(a) = 0.2a^2 - 6$ eller $f(xs) = 0.2xs^2 - 6$ til trods for at a allerede oprettet som enkeltvariabel i *Værdier*, og xs allerede er oprettet i som søjlevariabel i *Søjler*. Man siger, at den variabel, der er brugt til formulering af regneforskriften, er *lokal*.

Man kan godt i et program definere funktionen f ved blot at skrive $f = 0.2x^2 - 6$. Når man senere kalder f (dvs. lader f optræde i en programlinje), så vil f blive udregnet med den værdi x har i det øjeblik, der kaldes. Programmet vil imidlertid så ikke forstå meningen med f. eks. $f(3)$. Og det ville ikke forstå den måde, hvorpå vi ovenfor har benyttet $f(xstart)$ og $f(t)$.

```

Func f(x)
  Local t, n
  t := 2*x^2-3
  n:= 3*x-8
  Return t/n
Endfunc

```

I tilfælde af en funktion, der kræver flere linjer til sin definition, går man frem som vist her i marginen med

$$f(x) = \frac{2x^2-3}{3x-8}$$

som eksempel. Linjen *Local t, n* gør de to variable t og n lokale, således at de samme betegnelser frit kan bruges til andre ting andre steder i programmet.

Begrebet *Function* har en del lighedspunkter med begrebet *Procedure*, som vi diskuterer i afsnit 6.